Artificial Intelligence
AI 2002
Lecture 18

Mahzaib Younas

Lecturer Department of Computer Science

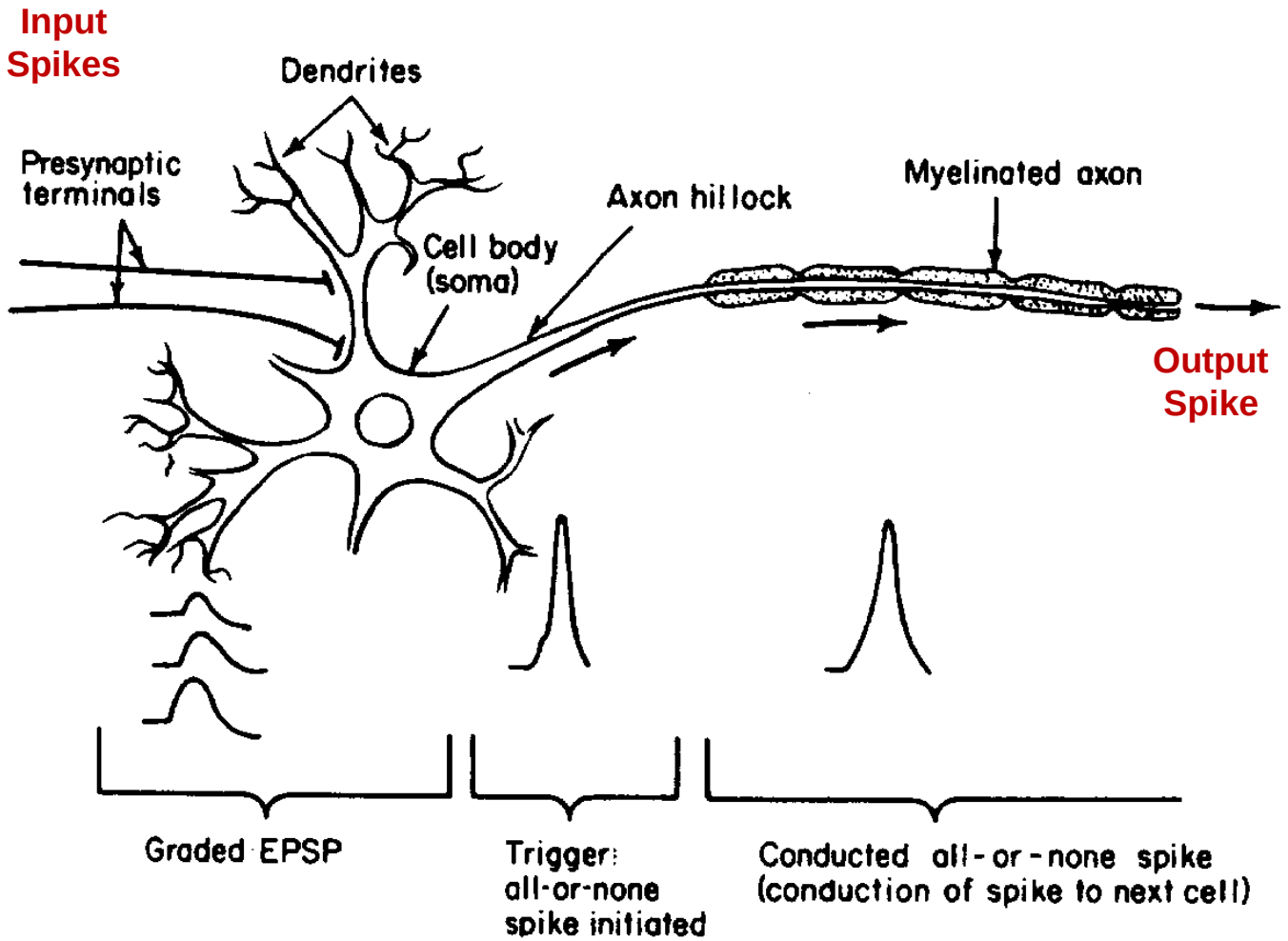
FAST NUCES CFD

Biological Inspiration

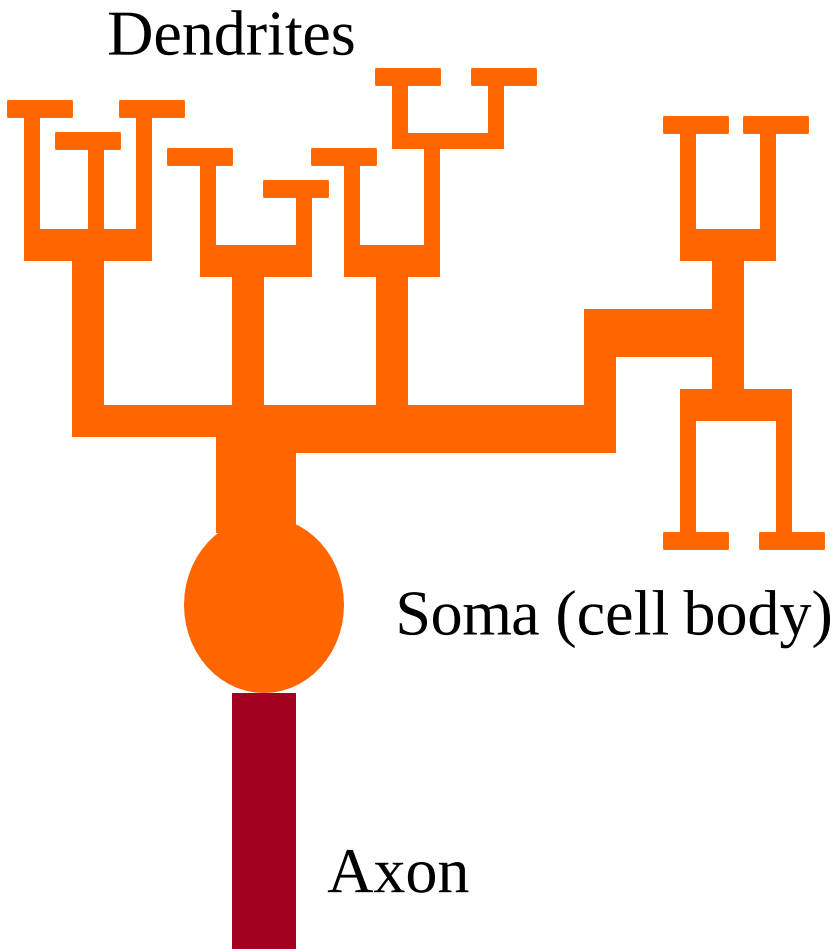
- Animals are able to **react adaptively to changes** in their external and internal environment, and they use their **nervous system** to perform these behaviours.

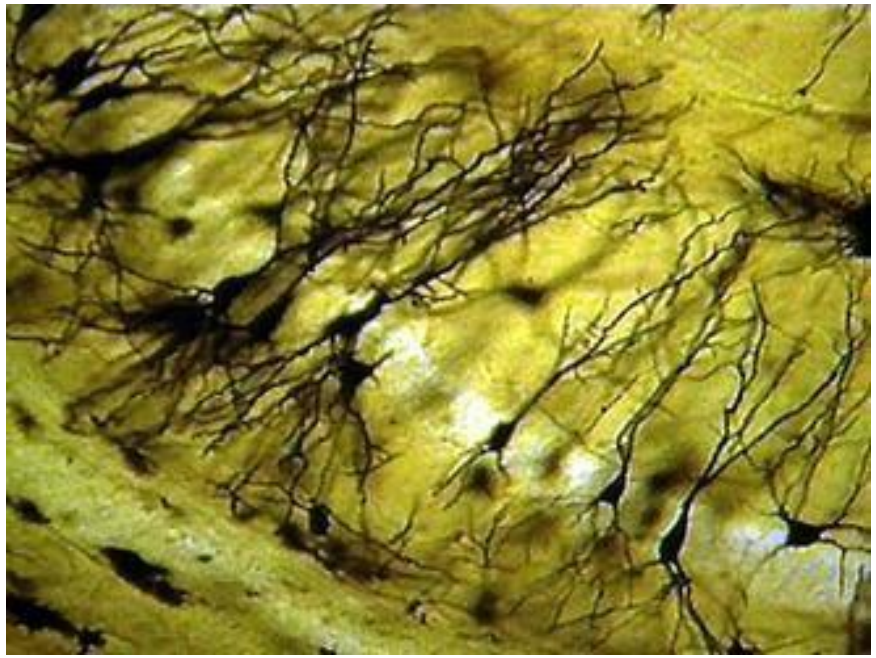
- An appropriate model/simulation of the nervous system should be able to produce similar responses and behaviours in artificial systems.

Biological Inspiration



Biological Inspiration



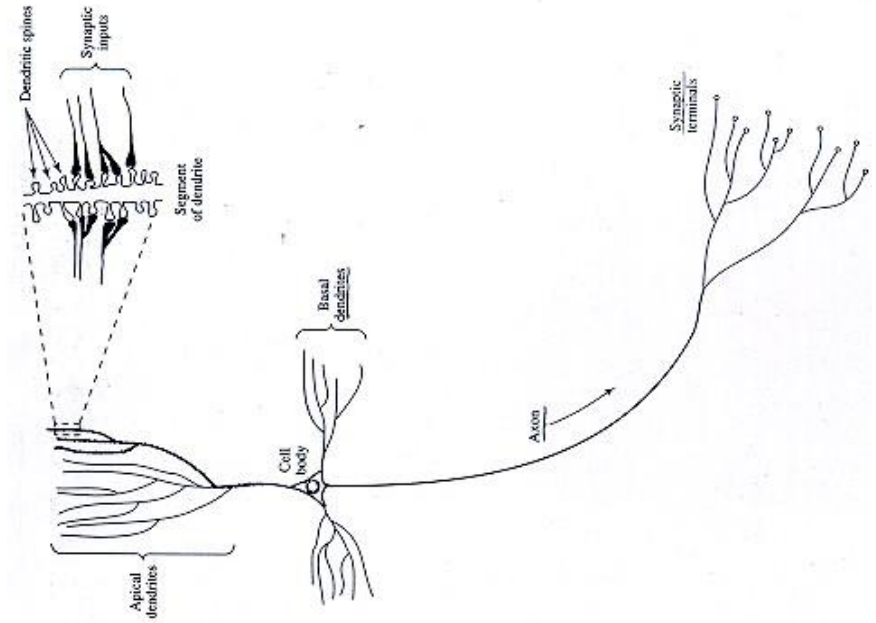


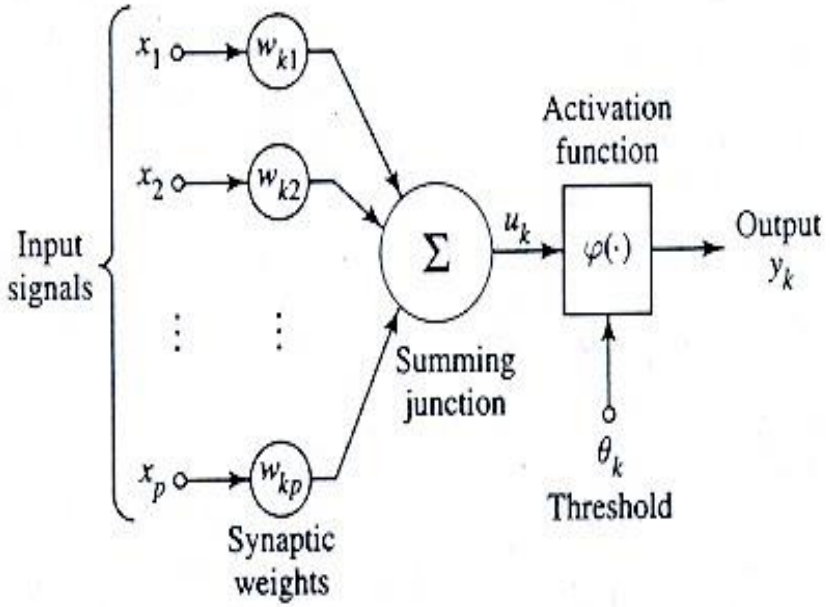
Biological Inspiration

Four Parts of Typical Nerve Cell:

- **Dendrites:**
  accepts the inputs
- **Soma:**
  process the inputs
- **Axon:**
  turns the process input into outputs
- **Synapses:**
  the electromechanical contact between the neurons

Biological Inspiration





Dendrite | Cell Body | Axon

ANN

A simplest type of ANN system is based on a unit called a perceptron.

- A perceptron
  - takes a vector of real-valued inputs,
  - calculates a linear combination of these inputs,
  - then outputs a 1 if the result is greater than some threshold and -1 otherwise.

$$O(x_1, \ldots\ldots\ldots x_n) = \begin{cases} 1 & if \quad w_o + w_1 x_1 + w_2 x_2 \ldots\ldots\ldots + w_n x_n > 0 \\ 0 & otherwise \end{cases}$$

ANN

- where each ***wi*** is a real-valued constant, or weight,
- that determines the contribution of input ***xi*** to the perceptron output.
- The quantity (***w0***) is a threshold
  - the weighted combination of inputs $w_1x_1 + w_2x_2 \ldots\ldots\ldots + w_nx_n$ must exceed in order for the perceptron to output a 1

$$O(x_1, \ldots\ldots\ldots x_n) = \begin{cases} 1 & if \quad w_o + w_1x_1 + w_2x_2 \ldots\ldots\ldots + w_nx_n > 0 \\ 0 & otherwise \end{cases}$$

ANN

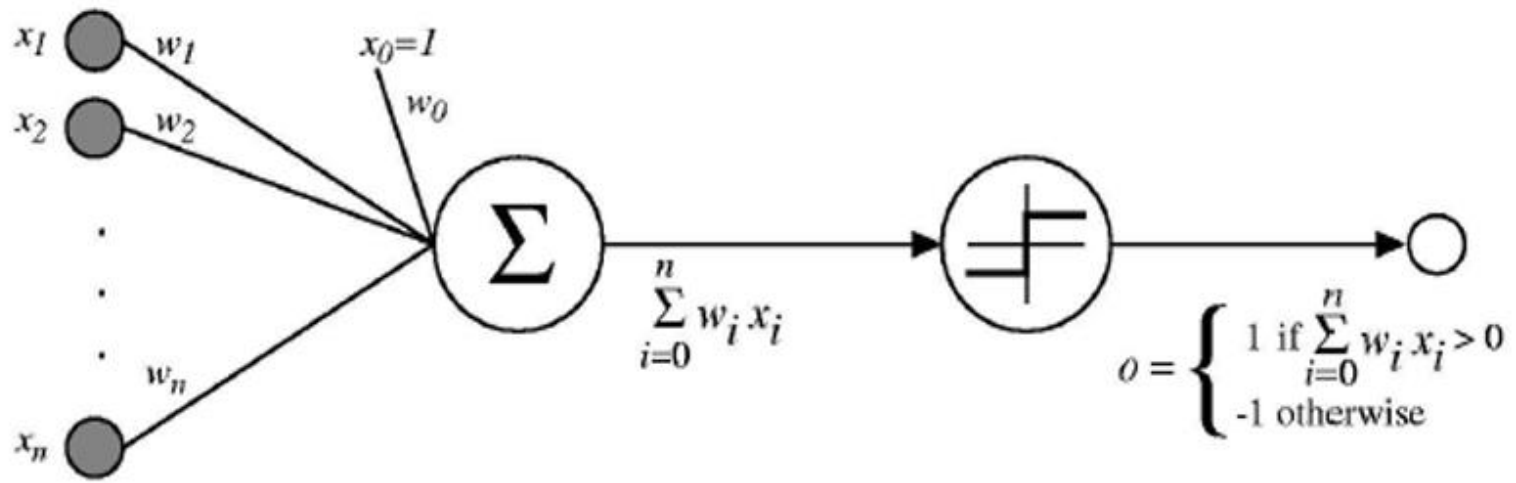
- We may imagine an additional constant input $\boldsymbol{x0} = \mathbf{1}$, allowing to write the above inequality as

$$\sum_{i=0}^{n} w_i x_i > 0$$

- or in vector form as

$$O(x_1, \ldots\ldots\ldots x_n) = \begin{cases} 1 \quad if \quad w_o . x_n > 0 \\ -1 \quad otherwise \end{cases}$$

Perceptron



$$o(x_1, \ldots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1 x_1 + \cdots + w_n x_n > 0 \\ -1 & \text{otherwise.} \end{cases}$$

Perceptron Example:

- Imagine a perceptron (in your brain). The perceptron tries to decide if you should go to a concert.
- Is the artist good? Is the weather good?
- What weights should these facts have?

Criteria	**Input**	**Weight**
Artist is Good.	X1 = 0 or 1	W1 = 0.7
Weather is Good.	X2 = 0 or 1	W2 = 0.6
Friend will come.	X3 = 0 or 1	W3 = 0.5
Food is served.	X4 = 0 or 1	W4 = 0.3
Juice is served.	X5 = 0 or 1	W5 = 0.4

Step 1:

- Set the threshold value

Threshold = 1.5

- Multiply all the inputs with its weight

x1 * w1 = 1 * 0.7 = 0.7

x2 * w2 = 0 * 0.6 = 0

x3 * w3 = 1 * 0.5 = 0.5

x4 * w4 = 0 * 0.3 = 0

x5 * w5 = 1 * 0.4 = 0.4

Step 2

- **Sum all the results:**

  0.7 + 0 + 0.5 + 0 + 0.4 = 1.6 (The Weighted Sum)

- **Activate the Output:**

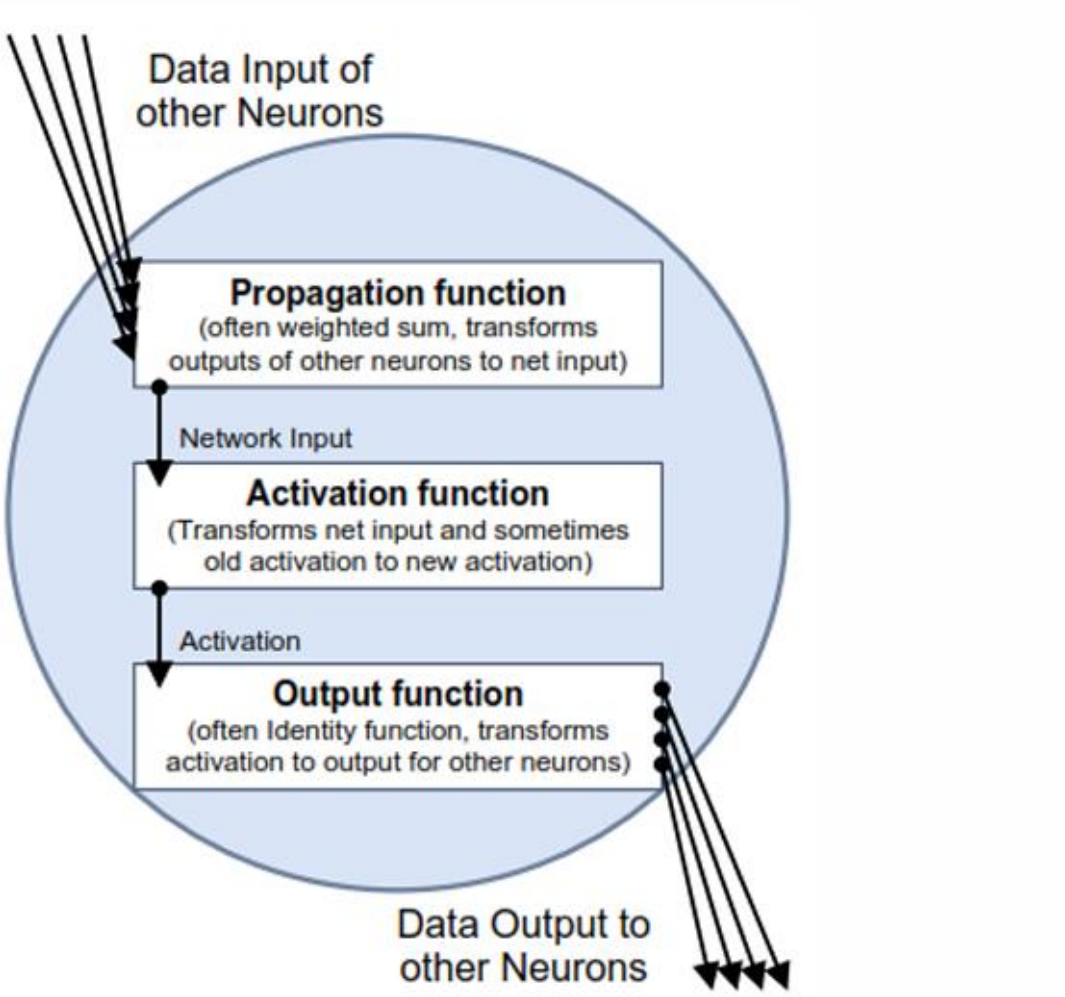
  Return true if the sum > 1.5 ("Yes I will go to the Concert")

Neural Network Components

Neural Network Components

- A neural network is a sorted triple $(\boldsymbol{N}, \boldsymbol{V}, \boldsymbol{w})$ with two sets N, V and a function w,
  - N is the set of neurons and
  - V is a sorted set $\{(i, j) | i, j \in N\}$ whose elements are called connections between neuron i and neuron j.
- The function $w : V \rightarrow R$ defines the weights, where as $w(i, j)$,
  - The weight of the connection between neuron i and neuron j, is shortly referred to as wi,j.

Neural Network Components



Input Neuron

- An input neuron is an identity neuron. It exactly forwards the information received.
  - Input neuron only forwards data

Thus, it represents the identity function which cab be indicated by the symbol /

- The input neuron is represented by the symbol

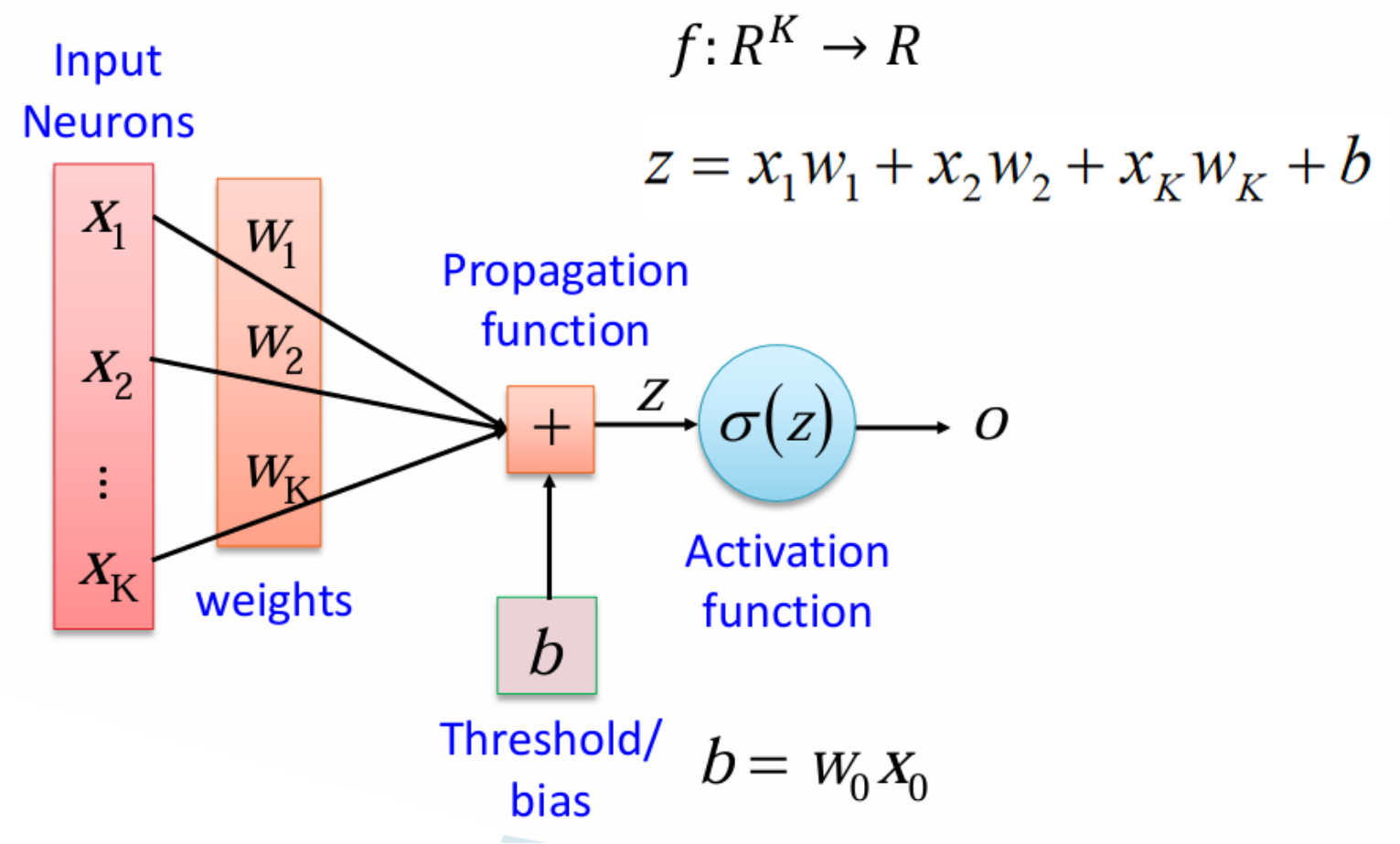
Binary Neuron

- Information processing neurons process the input information somehow, i.e. do not represent the identity function.
- A binary neuron sums up all inputs by using the weighted sum as propagation function which is illustrate by the sigma sign.

$$\sum$$

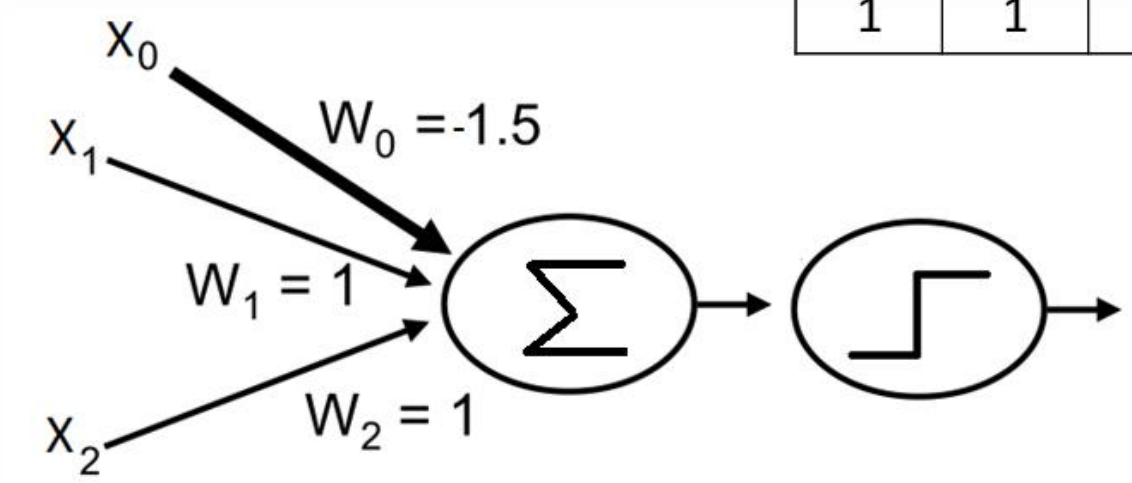
- The activation function of the neuron is also binary threshold function, which can be illustrated by ⌐‾

Neural Network Components



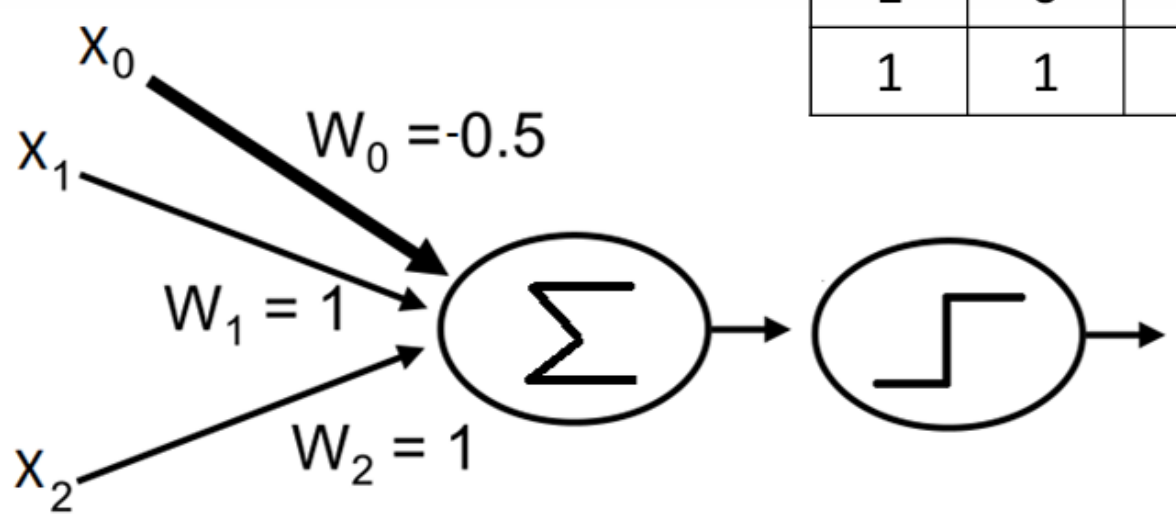
AND Function

X_1	X_2	Y
0	0	0
0	1	0
1	0	0
1	1	1



OR Function

X_1	X_2	Y
0	0	0
0	1	1
1	0	1
1	1	1



Perceptron Training Rule

- How to learn the weights for a single perceptron.
  - Begin with random weights,
  - Iteratively apply the perceptron to each training example,
  - Modifying the perceptron weights whenever it misclassifies an example.
  - This process is repeated, iterating through the training examples as many times as needed until the perceptron classifies all training examples correctly.
  - Weights are modified at each step according to the perceptron training rule.

Perceptron Training Rule

The ***perceptron training rule,*** which revises the weight w_i associated with input x_i according to the rule:

$$w_i \leftarrow w_i + \Delta w_i$$

where

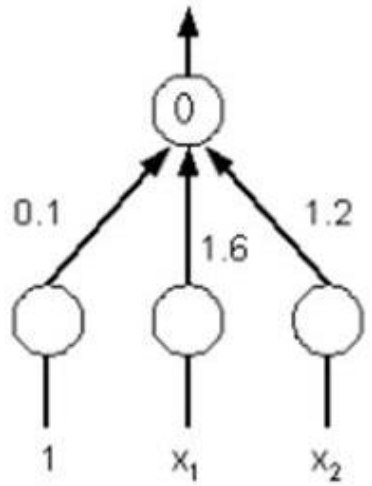
$$\Delta w_i = \eta(t - o)x_i$$

Where:

- t is target value
- o is perceptron output
- η is small constant (e.g., 0.1) called *learning rate*

Perceptron Training Rule

training set: $x_1 = 1, x_2 = 1 \rightarrow 1$ $\eta = 0.5$
$x_1 = 1, x_2 = -1 \rightarrow -1$
$x_1 = -1, x_2 = 1 \rightarrow -1$
$x_1 = -1, x_2 = -1 \rightarrow -1$



using these updated weights:

$x_1 = 1, x_2 = 1$: $0.1*1 + 1.6*1 + 1.2*1$	$= 2.9 \rightarrow 1$	OK
$x_1 = 1, x_2 = -1$: $0.1*1 + 1.6*1 + 1.2*-1$	$= 0.5 \rightarrow 1$	WRONG
$x_1 = -1, x_2 = 1$: $0.1*1 + 1.6*-1 + 1.2*1$	$= -0.3 \rightarrow -1$	OK
$x_1 = -1, x_2 = -1$: $0.1*1 + 1.6*-1 + 1.2*-1$	$= -2.7 \rightarrow -1$	OK

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta(t - o)x_i$$

First Iteration

X1	X2	$W_0x_0 + W_1X_1 + W_2X_2$ = perceptron output	Threshold output	Target
1	1	(1*0.1) + (1*1.6) +(1*1.2) = 2.9	1	Ok
1	-1	(1*0.1) + (1*1.6) +(-1*1.2) = 0.5	1	Wrong
-1	1	(1*0.1) + (-1*1.6) +(1*1.2) = - 0.3	-1	Ok
-1	-1	(1*0.1) + (-1*1.6) +(-1*1.2) = -2.7	-1	Ok

Now update the weights

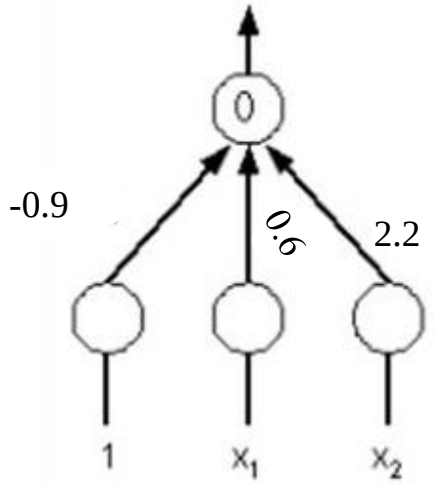
- Wrong output now we will update the weights

Wo	**W1**	**W2**
$\Delta w_i = n(t-o)x_i$ $\Delta w_0 = (0.5)(-1\ -1\ )(1)$ $\Delta w_0 = -1$	$\Delta w_i = n(t-o)x_i$ $\Delta w_1 = (0.5)(-1\ -1\ )(1)$ $\Delta w_1 = -1$	$\Delta w_i = n(t-o)x_i$ $\Delta w_2 = (0.5)(-1\ -1\ )(-1)$ $\Delta w_2 = 1$
$\Delta w_0 = 0.1 - 1 = -0.9$	$\Delta w_1 = 1.6 - 1 = 0.6$	$\Delta w_2 = 1.2 + 1 = 2.2$

Iteration 2:

- Now we have to perform the second iteration with updating weights

X1	X2	$W_0x_0 + W_1X_1 + W_2X_2$ = perceptron output	Threshold output	Target
1	1	(1*-0.9) + (1*0.6) +(1*2.2) = 1.9	1	Ok
1	-1	(1*-0.9) + (1*0.6) +(-1*2.2) = -2.5	-1	Ok
-1	1	(1*-0.9) + (-1*0.6) +(1*2.2) = 0.7	1	Wrong
-1	-1	(1*-0.9) + (-1*0.6) +(-1*2.2) = -3.7	-1	Ok



Weights Update

Wo	**W1**	**W2**
$\Delta w_i = n(t-o)x_i$ $\Delta w_0 = (0.5)(-1\ -1\ )(1)$ $\Delta w_0 = -1$	$\Delta w_i = n(t-o)x_i$ $\Delta w_1 = (0.5)(-1\ -1\ )(-1)$ $\Delta w_1 = 1$	$\Delta w_i = n(t-o)x_i$ $\Delta w_2 = (0.5)(-1\ -1\ )(1)$ $\Delta w_2 = -1$
$\Delta w_0 = -0.9 - 1 = \ -1.9$	$\Delta w_1 = 0.6 + 1 = 1.6$	$\Delta w_2 = 2.2 - 1 = 1.2$

Iteration 3

X1	X2	$W_0x_0 + W_1X_1 + W_2X_2$ = perceptron output	Threshold output	Target
1	1	(1*-1.9) + (1*1.6) +(1*1.2) = 0.9	1	Ok
1	-1	(1*-1.9) + (1*1.6) +(-1*1.2) = -1.5	-1	Ok
-1	1	(1*-1.9) + (-1*1.6) +(1*1.2) = -2.3	-1	OK
-1	-1	(1*-1.9) + (-1*1.6) +(-1*1.2) = -4.7	-1	Ok

Example:

- Consider the following dataset consisting of five data points with two features $x1$ and $x2$. There are two classes (No) and the (Yes). Consider the following assumptions and apply Perceptron Training Rule to learn a separating line for the given dataset.

x_1	x_2	Class
1	2	*No*
3	3	*No*
−3	2	Yes
3	−6	*No*
−4	−2	Yes

- Assumptions
  - The bias term ($x_0 w_0$) always equals to 0.
  - The learning rate: $\eta = 1$.
  - The initial weights: $w1 = -10$, $w2 = -10$ Yes
  - The sgn function may be used as an activation function

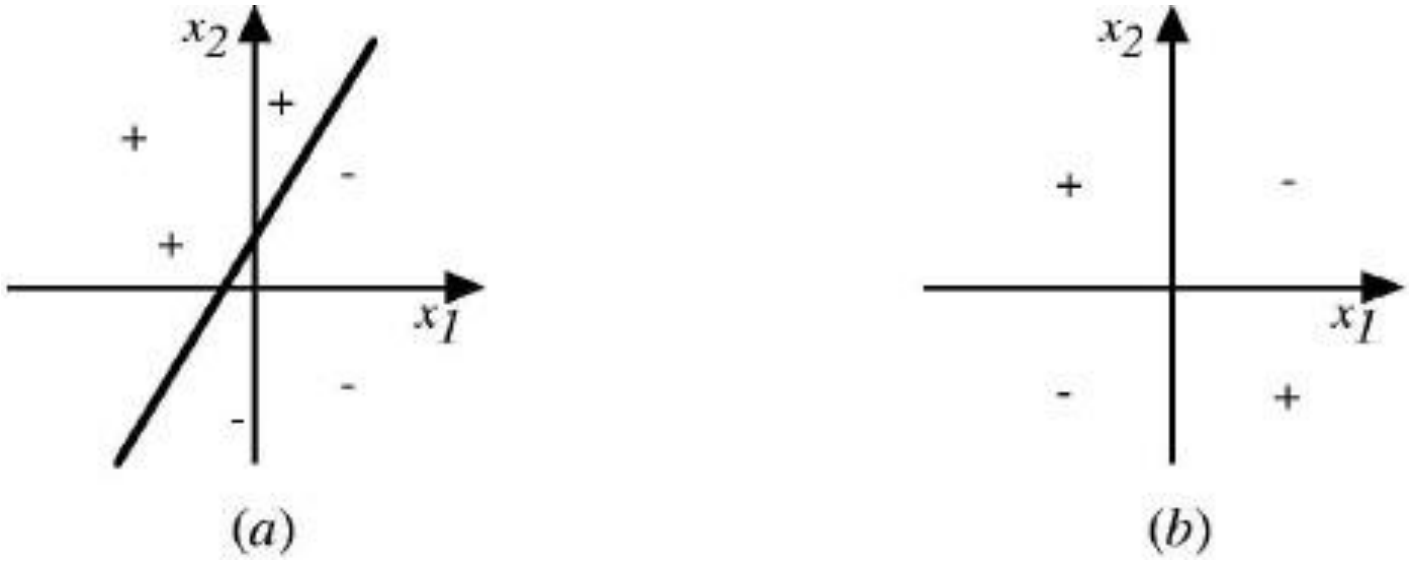
$$sgn(y) = \begin{cases} 1 & \text{if} \quad y > 0 \\ -1 & \text{otherwise} \end{cases}$$

Perceptron Training Rule Example

- The training rule will increase w, if $(t-o)$, η and xi are all positive.
  - if $xi = 0.8$, $\eta = 0.1$, $t = 1$, and $o=-1$, then the weight update will be
  - $\Delta wi = \eta(t-o)xi$
  - $=0.1(1-(-1))0.8=0.16.$

- On the other hand,
  - if $xi = 0.8$, $\eta = 0.1$, $t = -1$ and $o = 1$, then weights associated with positive xi will be decreased rather than increased.
  - $\Delta wi = \eta(t-o)xi$
  - $=0.1(-1-(1))0.8=-0.16.$

Perceptron



The **decision surface** represented by a **two-input perceptron x_1** and **x_2**. ***(a)*** A set of training examples and the decision surface of a perceptron that classifies them correctly. ***(b)*** A set of training examples that is not linearly separable.

Perceptron Training Rule

- The **perceptron rule** finds a successful weight vector when the training examples are **linearly separable**,
- It fails to converge if the examples are **not linearly separable**.
- The solution is … **Delta Rule** also known as **(Widrow- Hoff Rule)**
- **Delta Rule**
- use ***gradient descent*** to search the hypothesis space of possible weight vectors to find the weights that best fit the training examples.